

C# 8 New Features with Examples

C# 8.0, released with .NET Core 3.0 and Visual Studio 2019, introduced several new features and enhancements that aimed to improve the language's usability, readability, and safety. Some of the notable features include:

1. **Readonly Struct Members**
2. **Default Interface Methods**
3. **Pattern Matching Enhancements**
4. **Using Declarations**
5. **Static local functions**
6. **Disposable ref structs**
7. **Nullable reference types**
8. **Asynchronous streams**
9. **Asynchronous disposable**
10. **Indices and ranges**
11. **Null-coalescing assignment**
12. **Unmanaged constructed types**
13. **Stackalloc in nested expressions**
14. **Enhancement of interpolated verbatim strings**

Note: To work with C# 8 features, we will use Visual Studio 2019 and Creating .NET Core 3.1 Console Applications.

Readonly Struct in C# 8

In readonly structure, we declare the structure with the readonly modifier, and readonly structure indicates that the given structure is immutable. When you create a readonly structure, it is necessary to use a readonly modifier with its fields. If you do not do this, then the compiler will give an error. Even if you need to use only get accessors, you will get an error if you use set accessors.

For a better understanding, please have a look at the below example. In the below example, we declare the structure as readonly and the Height and Width fields as readonly with the get accessors only.

```
using System;
namespace Csharp8Features
{
    public readonly struct Rectangle
    {
        public readonly double Height { get; }
        public readonly double Width { get; }
        public double Area => (Height * Width);
        public Rectangle(double height, double width)
        {
            Height = height;
            Width = width;
        }
        public override string ToString()
    }
}
```

```

        {
            return $"(Total area for height: {Height}, width: {Width}) is {Area}";
        }
    }
}
class Program
{
    static void Main(string[] args)
    {
        Rectangle rectangle = new Rectangle(10, 20);
        Console.WriteLine("Height: " + rectangle.Height);
        Console.WriteLine("width: " + rectangle.Width);
        Console.WriteLine("Rectangle Area: " + rectangle.Area);
        Console.WriteLine("Rectangle: " + rectangle);
        Console.ReadKey();
    }
}
}

```

Output:

```

Height: 10
width: 20
Rectangle Area: 200
Rectangle: (Total area for height: 10, width: 20) is 200

```

For a detailed understanding of Readonly Struct in C#, please click the URL below.

<https://dotnettutorials.net/lesson/readonly-structs-in-csharp-8/>

Default Interface Methods in C# 8:

Before C# 8.0, interfaces only contained the declaration of the members (methods, properties, events, and indexers). But, from C# 8.0, it is allowed to add members and their implementation to the interface. Now, you can add a method with their implementation to the interface without breaking the existing interface implementation. Such types of methods are known as default interface methods (also known as the virtual extension methods).

The main benefit of the Default method is that it allows us to add new functionality to the interfaces of our libraries and ensure backward compatibility with code written for older versions of those interfaces. For a better understanding, please have a look at the below example.

```

using System;
namespace Csharp8Features
{
    interface IDefaultInterfaceMethod
    {
        public void DefaultMethod()
        {
            Console.WriteLine("I am a default method in the interface!");
        }
    }
}

```

```

    }
}
class AnyClass : IDefaultInterfaceMethod
{
}
class Program
{
    static void Main(string[] args)
    {
        IDefaultInterfaceMethod anyClass = new AnyClass();
        anyClass.DefaultMethod();
        Console.ReadKey();
    }
}
}

```

Output:

```
I am a default method in the interface!
```

Please click the URL below for a detailed understanding of Default Interface Methods in C#.

<https://dotnettutorials.net/lesson/default-interface-methods-csharp-8/>

Enhanced Pattern Matching in C# 8

C# 7.0 first introduced syntax for type and constant patterns using the `is` expression and the `switch` statement. These features represented the first tentative steps toward supporting programming paradigms where data and functionality live apart. Other language tools are needed as the industry moves toward more microservices and other cloud-based architectures.

C# 8.0 expands this vocabulary so you can use more pattern expressions in more places in your code. Consider these features when your data and functionality are separate. Consider pattern matching when your algorithms depend on a fact other than the runtime type of an object. These techniques provide another way to express designs.

In addition to new patterns in new places, C# 8.0 adds recursive patterns. Recursive patterns are patterns that can contain other patterns. In C# 8, the .Net Development team has introduced the following Patterns.

1. Switch Expressions
2. Property Patterns
3. Tuple Patterns
4. Positional Patterns

Please click the URL below for a detailed understanding of Enhanced Pattern Matching in C# 8.

<https://dotnettutorials.net/lesson/pattern-matching-in-csharp-8/>

Using Declarations in C# 8

With the new C# 8 “using declarations,” the code with the using statement can be simplified. Now, the curly brackets are no longer required. The Dispose method is also called automatically at the end of the method’s scope (at the end of the method body). The compiler also creates a *try/finally block to ensure the Dispose method is called. For a better understanding, please have a look at the below example.

```
using System;
namespace Csharp8Features
{
    public class UsingDeclarations
    {
        public static void Main()
        {
            using var resource = new Resource();
            resource.ResourceUsing();
            Console.WriteLine("Doing Some Other Task...");
        }
    }
    class Resource : IDisposable
    {
        public Resource()
        {
            Console.WriteLine("Resource Created...");
        }
        public void ResourceUsing()
        {
            Console.WriteLine("Resource Using...");
        }
        public void Dispose()
        {
            Console.WriteLine("Resource Disposed...");
        }
    }
}
```

Output:

```
Resource Created...
Resource Using...
Doing Some Other Task...
Resource Disposed...
```

Please click the URL below for a detailed understanding of Using Declarations in C# 8.

<https://dotnettutorials.net/lesson/using-declarations-csharp-8/>

Static Local Functions in C# 8

Local functions are introduced in C# 7. But in C# 7, using a static modifier with the local function is impossible, i.e., static local functions are not allowed. This feature is added in C# 8.0. From C# 8.0, we can use a static modifier with the local function. This ensures that the static local function does not reference any variable from the enclosing or surrounding scope. The compiler will throw an error if a static local function tries to access the variable from the enclosed scope. Let us understand this with an example.

```
using System;
namespace Csharp8Features
{
    public class LocalFunctions
    {
        public static void Main()
        {
            Calculate();
        }
        public static void Calculate()
        {
            int X = 20, Y = 30;
            CalculateSum(X, Y);
            static void CalculateSum(int Num1, int Num2)
            {
                int sum = Num1 + Num2;
                Console.WriteLine($"Num1 = {Num1}, Num2 = {Num2}, Result = {sum}");
            }
            CalculateSum(30, 10);
            CalculateSum(80, 60);
        }
    }
}
```

Output:

```
Num1 = 20, Num2 = 30, Result = 50
Num1 = 30, Num2 = 10, Result = 40
Num1 = 80, Num2 = 60, Result = 140
```

Please click the URL below for a detailed understanding of Static Local Functions in C# 8.

<https://dotnettutorials.net/lesson/static-local-functions-in-csharp-8/>

Disposable Ref Structs in C# 8

From C# 7.2 onward, a struct can be declared with the `ref` keyword. This allows the instances of ref structs to be allocated on the stack and restricts them from moving onto the managed heap. However, this also enforces some limitations over the ref structs, which cannot implement any interface.

In C# 8.0, a special exception to the above limitation was made for the `IDisposable` interface. Ref structs can now be disposable without implementing the `IDisposable` interface simply using a `Dispose` method. Let us understand this with an example.

```

using System;
namespace Csharp8Features
{
    public class DisposableRefStructs
    {
        public static void Main()
        {
            using var book = new Rectangle(10, 20);
            Console.WriteLine($"Area of Rectangle : {book.GetArea()}");
            Console.WriteLine("Main Method End");
        }
    }
    ref struct Rectangle
    {
        private double Height { get; set; }
        private double Width { get; set; }
        public Rectangle(double height, double width)
        {
            Height = height;
            Width = width;
        }
        public double GetArea()
        {
            return Height * Width;
        }
        public void Dispose()
        {
            Console.WriteLine("Rectangle Object Disposed Of");
        }
    }
}

```

Please click the URL below for a detailed understanding of Disposable Ref Structs in C# 8.

<https://dotnettutorials.net/lesson/disposable-ref-structs-in-csharp-8/>

Nullable Reference Types in C# 8

C# 8.0 allows us to specify whether a variable should be null and when it cannot be null. Based on these annotations, the compiler will warn you when you are potentially using a null reference or passing a null reference to a function that will not accept it.

In C#, a reference type refers to an object created on the heap memory. When the reference type variable does not point to any object, its value is null. Before C# 8.0, all reference types were nullable. Nullable reference types refer to a group of features introduced in C# 8.0 that you can use to minimize the likelihood that your code causes the runtime to throw `NullReferenceException`. Let us understand this with an example.

```

using System;
namespace Csharp8Features
{

```

```

public class NullableReferenceTypes
{
    public static void Main()
    {
        string message = null;
        // warning: dereference null.
        Console.WriteLine($"The length of the message is {message.Length}")
        var originalMessage = message;
        message = "Hello, World!";
        // No warning. Analysis determined "message" is not null.
        Console.WriteLine($"The length of the message is {message.Length}")
        // warning!
        Console.WriteLine(originalMessage.Length);
    }
}

```

Please click the URL below for a detailed understanding of Nullable Reference Types in C# 8.

<https://dotnettutorials.net/lesson/nullable-reference-types-in-csharp-8/>

Asynchronous Streams in C# 8

Async Streams is the new feature in C# 8.0, which provides async support for handling streams or IEnumerable data. In this article, we cover all the aspects of Async Streams (IAsyncEnumerable), including how to use ConfigureAwait and how to use CancellationToken. Cancellation token can be a great way to manage async programming in Dotnet core and C#, but with Async Streams, it can be a bit difficult and sometimes, if not used properly, can give errors. We shall be discussing briefly everything about AsyncStreams. IAsyncEnumerable also has options and support for CancellationToken as well as ConfigureAwait. Let us understand this with an example.

```

using System;
using System.Threading.Tasks;
namespace Csharp8Features
{
    public class NullableReferenceTypes
    {
        static async Task Main(string[] args)
        {
            await foreach (var number in GenerateSequence())
            {
                Console.WriteLine(number);
            }
        }
        public static async System.Collections.Generic.IAsyncEnumerable<int>
GenerateSequence()
        {
            for (int i = 0; i < 20; i++)
            {
                await Task.Delay(100);
            }
        }
    }
}

```

```

        yield return i;
    }
}
}

```

Please click the URL below for a detailed understanding of Asynchronous Streams in C# 8.

<https://dotnettutorials.net/lesson/asynchronous-streams-in-csharp-8/>

Asynchronous Disposable in C# 8

Starting with C# 8.0, the language supports asynchronous disposable types that implement the `IAsyncDisposable` interface. You use the `await using` statement to work with an asynchronously disposable object. Let us understand this C# 8 new Feature with an example.

```

using System;
using System.Threading.Tasks;
using System.IO;
namespace Csharp8Features
{
    class AsynchronousDisposable
    {
        static async Task Main(string[] args)
        {
            await using (var disposableObject = new Sample())
            {
                Console.WriteLine("Welcome to C#.NET");
            } // DisposeAsync method called implicitly
            Console.WriteLine("Main Method End");
        }
    }
    public class Sample : IAsyncDisposable
    {
        static readonly string filePath = @"D:\MyTextFile1.txt";
        private TextWriter? textWriter = File.CreateText(filePath);
        public async ValueTask DisposeAsync()
        {
            if (textWriter != null)
            {
                textWriter = null;
            }
            await Task.Delay(1000);
            Console.WriteLine("DisposeAsync Clean-up the Memory!");
        }
    }
}

```

Please click the URL below for a detailed understanding of Asynchronous Disposable in C# 8.

Indices and Ranges in C# 8

As we already know about the Range and Indices. We use them several times in our programs. They provide a short syntax to represent or access a single or a range of elements from the given sequence or collections. The Range and Indexes make the C# syntax simpler and more readable.

Ranges and Indices in C# allow more natural syntax for accessing single items or ranges in a sequence. This language support relies on two new types and two new operators. Let us understand this C# 8 new Features with an example.

```
using System;
using System.Threading.Tasks;
namespace Csharp8Features
{
    class IndicesAndRanges
    {
        static async Task Main(string[] args)
        {
            var countries = new string[]
            {
                "INDIA",
                "USA",
                "UK",
                "NZ",
                "CANADA",
                "CHINA",
                "NEPAL",
                "RUSIA",
                "SRILANKA",
                "INDONESIA"
            };
            Index i1 = 4;
            Console.WriteLine($"{countries[i1]}"); // Output: "CANADA"
            // Index 4 from end of the collection
            Index i2 = ^4;
            Console.WriteLine($"{countries[i2]}"); // Output: "NEPAL"
        }
    }
}
```

Please click the URL below for a detailed understanding of Indices and Ranges in C# 8.

<https://dotnettutorials.net/lesson/indices-and-ranges-in-csharp-8/>

Null-Coalescing Assignment Operator in C# 8

C# 8.0 introduces the null-coalescing assignment operator `??=`. We can use this `??=` operator in C# to assign the value of its right-hand operand to its left-hand operand only if the left-hand operand evaluates to null. That means the null-coalescing assignment operator `??=` assigns a variable only if it is null. Let us understand this C# 8 new Features with an example.

```
using System;
using System.Threading.Tasks;
namespace Csharp8Features
{
    class NullCoalescingAssignment
    {
        static async Task Main(string[] args)
        {
            int? Age = null;
            Age ??= 20;
            Console.WriteLine(Age);
        }
    }
}
```

Please click the URL below for a detailed understanding of the Null-Coalescing Assignment Operator in C# 8.

<https://dotnettutorials.net/lesson/null-coalescing-assignment-operator-in-csharp-8/>

Unmanaged Constructed Types in C# 8

A type is called constructed type if it is generic and the type parameter is already defined, such as `List<string>`, `List`, etc. In C# 7.3 and earlier, a constructed type (including at least one type of argument) can't be unmanaged. Starting with C# 8.0, a constructed value type is unmanaged if it only contains fields of unmanaged types.

Beginning with C# 7.3, you can use the unmanaged constraint to specify that a type parameter is a non-pointer, non-nullable unmanaged type. Beginning with C# 8.0, a constructed struct type that contains fields of unmanaged types only is also unmanaged. Let us understand this C# 8 new Feature with an example.

```
using System;
namespace Csharp8Features
{
    public struct Coords<T>
    {
        public T X;
        public T Y;
    }
    public class UnmanagedType
    {
        public static void Main()
        {
            DisplaySize<Coords<int>>>();
            DisplaySize<Coords<double>>>();
        }
    }
}
```

```

        private unsafe static void DisplaySize<T>() where T : unmanaged
        {
            Console.WriteLine($"{typeof(T)} is unmanaged and its size is {sizeof(T)} bytes");
        }
    }
}

```

Please click the URL below for a detailed understanding of Unmanaged Constructed Types in C# 8.

<https://dotnettutorials.net/lesson/unmanaged-constructed-types-in-csharp-8/>

Stackalloc in Nested Expressions in C# 8

The stackalloc operator in C# allocates a block of memory in the stack. The memory block will be created during the execution of the method, and it is automatically deleted when the method is returned. You cannot explicitly free the memory allocated with stackalloc. A stack-allocated memory block is not subject to garbage collection and must not be pinned with a fixed statement. Let us understand this with an example.

```

using System;
namespace Csharp8Features
{
    public class StackMemoryAllocation
    {
        public static void Main()
        {
            Span<int> set = stackalloc[] { 1, 2, 3, 4, 5, 6 };
            var subSet = set.Slice(3, 2);
            foreach (var n in subSet)
            {
                Console.WriteLine(n); // Output: 4 5
            }
        }
    }
}

```

Please click the URL below for a detailed understanding of Stackalloc in Nested Expressions in C# 8.

<https://dotnettutorials.net/lesson/stackalloc-in-nested-expressions-in-csharp-8/>

In the next article, I will discuss [Readonly Structs in C# 8](#) with Examples. In this article, I try to explain **C# 8 New Features** with Examples. I hope you enjoy this C# 8 New Features with Examples article.